



INSTITUT UNIVERSITAIRE SAINT JEAN
SAINT JEAN INGÉNIEUR

CAHIER DE CHARGES

IMPLÉMENTATION DE LA COMMANDE GREP EN LANGAGE C

MEMBRES DE L'EQUIPE 7 (PROJET MYGREP)^o

- ▶ ALOBWEDE Lesnar Ahone
- ▶ EKANJE VANESSA EWOSE
- ▶ TETEYA TOLEQUE Etienne Dieu beni
- ▶ TSALA Yannick Bertrand
- ▶ NINKAM YOUNBISSI Noémie Amanda

CAHIER DE CHARGES

- ▶ Introduction
- ▶ Objectifs du projet
- ▶ Présentation de la commande(définition, utilité, périmètre fonctionnel)
- ▶ Expression du besoin
- ▶ Fonctionnalités attendues
- ▶ Contraintes
- ▶ Livrables
- ▶ Plan de travail

INTRODUCTION

De nombreux étudiants et professionnels se contentent d'utiliser les outils systèmes mais peu nombreux sont ceux qui comprennent la logique qui s'y cache en arrière-plan. Ainsi, dans un souci de compréhension des systèmes d'exploitation par la pratique, il nous a été consigné, à nous, étudiants en 3^{ème} année Systèmes Réseaux et Télécommunications, d'implémenter la commande **grep** (commande du système Linux) en langage C. Il s'agit de notre commande **mygrep**.

PRÉSENTATION DE LA COMMANDE `grep`

Définition

La commande `grep` (Global Regular Expression Print) est un utilitaire des systèmes Linux et Unix permettant de rechercher des motifs dans un ou plusieurs fichiers. Elle parcourt le fichier ligne par ligne et affiche celles qui correspondent au motif recherché. Le résultat affiché peut être modifié dépendamment des options utilisées

PRÉSENTATION DE LA COMMANDE grep

Utilité

Généralement, la commande grep sert à

- La recherche rapide d'informations dans des fichiers texte ;
- Le filtrage de résultats d'autres commandes ;
- L'analyse de journaux (logs) ;
- La détection des tentatives d'intrusion.

PRÉSENTATION DE LA COMMANDE `grep`

Utilité (quelques cas de figures)

`grep "error" syslog.txt` permet de rechercher les lignes d'erreur dans un fichier de logs (ici `syslog.txt`)

PRÉSENTATION DE LA COMMANDE `grep`

Utilité (quelques cas de figures)

`grep "Failed password" auth.log` permet de rechercher les lignes d'erreur dans un fichier de logs (ici `syslog.txt`)

PRÉSENTATION DE LA COMMANDE `grep`

Utilité (quelques cas de figures)

```
grep -E "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"  
utilisateurs.txt
```

permet de récupérer les adresses électroniques dans un le fichier utilisateurs.txt

PRÉSENTATION DE LA COMMANDE grep

Comment marche la fonction grep ?

Lors de l'exécution de la commande grep, trois entités interviennent:

- Le terminal
- Le shell
- Le noyau (kernel)

PRÉSENTATION DE LA COMMANDE `grep`

Comment marche la fonction `grep` ?

Lorsque l'utilisateur entre la ligne de commande dans le terminal, par exemple (`grep « network » config.txt`), le terminal transmet la ligne au shell et voici ce qui se passe en arrière-plan:

PRÉSENTATION DE LA COMMANDE grep

Comment marche la fonction grep ?

- Analyse de la syntaxe et parsing : le shell découpe la ligne en morceaux (tokens) : la **commande (grep)**, les **options** (il n'y en a pas ici), les **arguments** (motif recherché et fichier où on recherche).

PRÉSENTATION DE LA COMMANDE `grep`

Comment marche la fonction `grep` ?

- **Recherche de l'exécutable:** le shell vérifie si la commande est intégrée (built-in) ou s'il s'agit d'un programme externe (situé dans les dossiers prévus comme `/usr/bin` ou `/bin`). Dans le cas de `grep`, son programme externe est stocké dans le dossier `/usr/bin/grep`.

PRÉSENTATION DE LA COMMANDE grep

Comment marche la fonction grep ?

- Appels systèmes et création de processus : une fois le fichier exécutable trouvé, le shell enverra ce fichier au noyau pour qu'il puisse traiter le fichier exécutable. Cette demande est une suite d'appels système

PRÉSENTATION DE LA COMMANDE grep

Comment marche la fonction grep ?

- Appels systèmes et création de processus :

Les appels système faits sont

1.fork() pour La création du processus : le shell demande au noyau de dupliquer l'environnement (shell actuel) pour créer un processus fils. Ainsi, le kernel devra allouer un espace mémoire pour le processus fils et lui allouer un PID unique. C'est là que sera traitée grep.

PRÉSENTATION DE LA COMMANDE grep

Comment marche la fonction grep ?

- Appels systèmes et création de processus :

Les appels système faits sont

2.execve() pour le chargement du programme: le kernel va récupérer le fichier exécutable de grep sur le disque dur, libérer la mémoire de l'ancien clone, charge le programme de grep à la place et initialise les arguments (on les place dans la pile du nouveau processus i.e. initialiser argc et argv)

PRÉSENTATION DE LA COMMANDE grep

Comment marche la fonction grep ?

- Appels systèmes et création de processus :

Les appels système faits sont

3.open() et read() pour l'accès aux données: le kernel vérifie si on a le droit de lire le fichier (ici conf.txt). Si on a le droit, le noyau va piloter le disque dur pour copier le contenu du fichier du DD vers l'espace mémoire alloué à grep.

PRÉSENTATION DE LA COMMANDE grep

Comment marche la fonction grep ?

- Appels systèmes et création de processus :

Les appels système faits sont

4.write() pour l'affichage du résultat: le kernel prend le résultat qui se trouve dans la mémoire du shell qui exécute grep et l'envoie vers la sortie standard (le terminal) pour que ça s'affiche à l'écran.

PRÉSENTATION DE LA COMMANDE grep

Comment marche la fonction grep ?

- Appels systèmes et création de processus :

Les appels système faits sont

5.exit() et wait() : `exit()` s'effectue au niveau de grep et signifie que grep a terminé son travail et s'arrête. Le kernel libère toute la mémoire utilisée par grep. Ensuite, il réveille le shell parent (qui était en attente avec le `wait()`) et lui transmet le code retour. Le shell réaffiche ainsi l'invite de commande (\$)

EXPRESSION DU BESOIN

Il est question de mettre en place un outil en ligne de commande offrant des fonctionnalités de filtrage, de sélection et d'affichage des résultats conformes au comportement de la commande Linux grep. Elle permettra à un utilisateur de rechercher efficacement des motifs textuels simples ou complexes dans un ou plusieurs fichiers.

OBJECTIFS DU PROJET

- L'objectif principal de notre projet est de mettre sur pied un clone fonctionnel de la commande grep. Réaliser ce projet sera une occasion pour nous, une occasion de:
- Mieux nous initier à la programmation système sous Linux
- Comprendre comment marchent les appels systèmes, les mémoires (pour l'allocation)
- **DANS LE BUT FINAL** de comprendre le noyau Linux

PÉRIMÈTRE FONCTIONNEL

Nous parlerons ici des différentes fonctionnalités de la commande originale grep (Global Regular Expression Print), réparties en 8 catégories:

- Informations générales sur la commande (Generic Program Information)
- Syntaxe des motifs (Pattern Syntax)
- Contrôle de correspondance (Matching Control)
- Contrôle général de la sortie (General Output Control)
- Contrôle des préfixes de ligne de sortie (Output Line Prefix Control)
- Contrôle des lignes de contexte (Context Line Control)
- Sélection de fichiers et de répertoires (File and Directory Selection)

PÉRIMÈTRE FONCTIONNEL

INFORMATIONS GÉNÉRALES SUR LA COMMANDE (GENERIC PROGRAM INFORMATION)

- ▶ **--help** : affiche le message d'utilisation et exit
- ▶ **--version (-V)** : affiche le numéro de version de la commande grep et exit

PÉRIMÈTRE FONCTIONNEL

SYNTAXE DES MOTIFS (PATTERN SYNTAX)

- ▶ **--extended-regexp (-E)** : interprète le motif recherché comme une expression régulière étendue (ERE) ;
- ▶ **--fixed-strings (-F)** : les motifs recherchés sont vus comme des chaînes de caractères fixes, pas des expressions régulières ;
- ▶ **--basic-regexp (-G)** : interprète le motif recherché comme une expression régulière basique (BRE) ;
- ▶ **--perl-regexp (-P)** : interprète les motifs comme des expressions régulières perl-compatibles.

PÉRIMÈTRE FONCTIONNEL

CONTRÔLE DE CORRESPONDANCE (MATCHING CONTROL)

- ▶ **-e** *MOTIF* (ou **--regexp=** *MOTIF*) : *MOTIF* est ce qu'il faut rechercher, option idéale quand le motif commence par un tiret (-) ;
- ▶ **-f** *FILE* (ou **--file=***FILE*) : Obtenir les motifs à partir du fichier *FILE*, à raison d'un par ligne
- ▶ **-i** ou (**--ignore-case**) : ignorer la casse, *HouSE*=*house*=*hOuSe* ;
- ▶ **--no-ignore-case** : comportement par défaut du shell, *HouSE* != *house* != *hOuSe*;

PÉRIMÈTRE FONCTIONNEL

CONTRÔLE DE CORRESPONDANCE (MATCHING CONTROL)

- ▶ **--invert-match (-v)** : les lignes affichées sont celles qui ne contiennent pas le(s) motif(s) recherché(s) ;
- ▶ **--word-regexp(-w)** : Sélectionner uniquement les lignes contenant des correspondances qui forment des mots entiers ;
- ▶ **--line-regexp(-x)** : Sélectionner uniquement les lignes qui correspondent exactement à la ligne entière

PÉRIMÈTRE FONCTIONNEL

CONTRÔLE GÉNÉRAL DE LA SORTIE (GENERAL OUTPUT CONTROL)

- ▶ **--count(-c)** : affiche le nombre de lignes du fichier qui ont le motif recherché ;
- ▶ **--color[WHEN]** ou **--colour[WHEN]** : Entourer les chaînes, lignes correspondantes ;
- ▶ **--files-without-match (-L)** :
- ▶ **--files-with-matches (-l)** : afficher uniquement le nom des fichiers contenant le motif.

PÉRIMÈTRE FONCTIONNEL

CONTRÔLE GÉNÉRAL DE LA SORTIE (GENERAL OUTPUT CONTROL)

- ▶ **-m *NUM*** ou **--max-count=*NUM*** : Arrêter la lecture d'un fichier après *NUM* lignes correspondantes;
- ▶ **--only-matching (-o)** : N'afficher que les parties correspondantes (non vides) d'une ligne correspondante ;
- ▶ **--quiet** ou **--silent** ou **-q** : mode silencieux ; ne rien écrire sur la sortie standard;
- ▶ **--no-messages (-s)** : Supprimer les messages d'erreur concernant les fichiers inexistants ou illisibles

PÉRIMÈTREFONCTIONNEL

CONTRÔLE DES PRÉFIXES DE LIGNE DE SORTIE (OUTPUT LINE PREFIX CONTROL)

- ▶ **--byte-offset (-b)** : afficher le décalage d'octets (en commençant à compter à partir de 0) au sein du fichier d'entrée avant chaque ligne de sortie
- ▶ **--with-filename (-H)** : Afficher le nom du fichier pour chaque correspondance
- ▶ **--no-filename(-h)** : Pas de nom de fichier en préfixe sur la sortie
- ▶ **--label=LABEL** : Afficher l'entrée provenant réellement de l'entrée standard comme si elle provenait du fichier LABEL
- ▶ **--line-number(-n)** : affiche le numéro des lignes où le motif recherché a été retrouvé
- ▶ **--initial-tab (-T)** : S'assurer que le premier caractère du contenu réel de la ligne se trouve sur un taquet de tabulation pour un alignement normal
- ▶ **--null (-Z)** : générer un octet de valeur zéro à la place du caractère qui suit normalement un nom de fichier

PÉRIMÈTRE FONCTIONNEL

CONTRÔLE DES LIGNES DE CONTEXTE (CONTEXT LINE CONTROL)

- ▶ **--after-context= *NUM*** ou **-A *NUM***: Afficher *NUM* lignes de contexte qui suivent (après) les lignes correspondantes
- ▶ **--before-context= *NUM*** ou **-B *NUM*** : Afficher *NUM* lignes de contexte qui précèdent (avant) les lignes correspondantes
- ▶ **--context= *NUM*** ou **-C *NUM*** : : Afficher *NUM* lignes de contexte en sortie (avant et après)
- ▶ **--group-separator= *SEP*** : Lorsque les options **-A**, **-B** ou **-C** sont utilisées, afficher *SEP* au lieu de **--** entre les groupes de lignes
- ▶ **--no-group-separator** : Lorsque les options **-A**, **-B** ou **-C** sont utilisées, ne pas afficher de séparateur entre les groupes de lignes

PÉRIMÈTRE FONCTIONNEL

SÉLECTION DE FICHIERS ET DE RÉPERTOIRES (FILE AND DIRECTORY SELECTION)

- ▶ **--text (-a)** : permet de traiter un fichier binaire comme si c'était un fichier texte.
- ▶ **--binary-files=TYPE**
- ▶ **--devices=ACTION** ou **-D ACTION** : Si un fichier d'entrée =périphérique, un FIFO, un socket, utiliser ACTION pour le traiter. **Valeurs de ACTION** : skip, read
- ▶ **--directories=ACTION** ou **-d ACTION** : Si un fichier d'entrée est un répertoire, utiliser ACTION pour le traiter. **Valeurs de ACTION** : skip, read, recurse

PÉRIMÈTRE FONCTIONNEL

SÉLECTION DE FICHIERS ET DE RÉPERTOIRES (FILE AND DIRECTORY SELECTION)

- ▶ **--exclude=***GLOB* : Ignorer tout fichier de la ligne de commande dont le suffixe du nom correspond au motif *GLOB*
- ▶ **--exclude-from=***FILE* : Ignorer les fichiers dont le nom de base correspond à l'un des motifs de noms de fichiers lus à partir de *FILE*
- ▶ **--exclude-dir=***GLOB* : Ignorer tout répertoire de la ligne de commande dont le suffixe du nom correspond au motif *GLOB*
- ▶ **-I** : traiter un fichier binaire comme s'il ne contenait pas de données correspondantes

PÉRIMÈTRE FONCTIONNEL

SÉLECTION DE FICHIERS ET DE RÉPERTOIRES (FILE AND DIRECTORY SELECTION)

- ▶ **--include=*GLOB*** Rechercher uniquement dans les fichiers dont le nom de base correspond à *GLOB* ;
- ▶ **--recursive (-r)** : lire tous les fichiers sous chaque répertoire, de manière récursive;
- ▶ **--dereference-recursive (-R):**

PÉRIMÈTRE FONCTIONNEL

AUTRES OPTIONS (OTHER OPTIONS)

- ▶ **--line-buffered** : Utiliser la mise en mémoire tampon par ligne (line buffering) sur la sortie. Cela peut entraîner une baisse des performances
- ▶ **--binary (-U)** : traiter le(s) fichier(s) comme binaire(s)
- ▶ **--null-data (-z)** : Traiter les données d'entrée et de sortie comme des séquences de lignes, chacune se terminant par un octet nul au lieu d'un saut de ligne

FONCTIONNALITÉS ATTENDUES

Toutes les options mentionnées ci haut sont attendues dans le projet mygrep.

CONTRAINTES

- ▶ **Langage de programmation : C;**
- ▶ La commande mygrep doit respecter le comportement de la commande originale grep ;
- ▶ La commande mygrep doit **fonctionner sur les systèmes Linux;**
- ▶ Le respect des délais.

PLAN DE TRAVAIL

Phase	Dure estimée	Description
Étude et analyse du projet	3 semaines (23 mars-13 avril)	Recherche des ressources à utiliser, étude de l'environnement de développement du projet, rédaction du cahier de charges et du début du dossier d'analyse et de conception
Conception et développement du projet	6 semaines (15 avril-20 mai)	Rédaction de la fonction de base (grep minimal), rédaction des fonctions correspondant aux options les plus utilisées fin de rédaction du dossier d'analyse et de conception
Test et améliorations	2 semaines (22 mai-7 juin)	Implémentation des options plus complexes (expressions régulières)
Finalisation et publication	1 semaine (19 mai au 23 mai)	Publication sur GitHub et documentation du projet

LIVRABLES ATTENDUS

- ▶ Cahier de charges ;
- ▶ Dossier de conception et d'analyse ;
- ▶ Présentation et démonstration ;
- ▶ Rapport technique ;
- ▶ Code source à publier sur GitHub



INSTITUT UNIVERSITAIRE SAINT JEAN
SAINT JEAN INGÉNIEUR

DOSSIER D'ANALYSE ET DE CONCEPTION IMPLÉMENTATION DE LA COMMANDE GREP EN LANGAGE C

DOSSIER D'ANALYSE ET DE CONCEPTION

- ▶ Introduction
- ▶ I. Analyse
- ▶ II. Conception
- ▶ Conclusion

INTRODUCTION

L'analyse du projet consiste à étudier le fonctionnement de la commande **grep** afin de préparer l'implémentation de **mygrep** en langage C. Elle permet d'identifier les besoins, les acteurs, les données et les traitements nécessaires au développement de la solution.

DOSSIER D'ANALYSE ET DE CONCEPTION

Analyse

- ▶ Portée du système (fonctionnalités couvertes et limites)
- ▶ Description du système
- ▶ Acteurs du système
- ▶ Expressions régulières
- ▶ Diagramme des cas d'utilisation

PORTÉE DU SYSTÈME

La commande mygrep que nous mettrons en place, devra fonctionner sur les systèmes Linux obligatoirement. Et fonctionner comme une commande

BESOINS FONCTIONNELS DU PROJET

En dehors du fait que notre solution mygrep doit avoir toutes les fonctionnalités de la commande de base grep, mygrep doit également se comporter comme une commande c'est-à-dire pouvoir être lancée depuis n'importe quel dossier du système selon les permissions qui sont accordées à l'utilisateur.

BESOINS FONCTIONNELS DU PROJET

Pour cela, on devra ajouter le fichier exécutable (binaire) du programme mygrep dans la variable environnement `$ PATH`. Pour y parvenir, nous devons créer le fichier binaire du programme mygrep en compilant tous les fichiers `.c` qui interviendront dans notre projet. La commande pour compiler est `gcc` et l'option `-o`.

BESOINS FONCTIONNELS DU PROJET

Après avoir obtenu le fichier exécutable, on devra l'ajouter dans le dossier `/usr/local/bin` avec la commande :

`sudo cp mygrep /usr/local/bin`. Après avoir entré cette commande, l'utilisateur devra entrer son mot de passe. Notre commande `mygrep` est installée et prête à être utilisée.

UTILISATEURS DE LA SOLUTION

Notre solution **mygrep** sera destinée à n'importe quel utilisateur du noyau Linux : des débutants jusqu'aux experts. Tant que votre machine fonctionne sur un système à noyau linux, notre solution est faite pour vous.

EXPRESSIONS RÉGULIÈRES

Définition

Une **expression régulière** (ou regex) est une séquence de caractères qui définit un modèle de recherche pour correspondre à des chaînes de caractères dans un texte. En termes simples, il s'agit d'un outil permettant de trouver, vérifier ou manipuler des fragments de texte en se basant sur des motifs préalablement définis.

EXPRESSIONS RÉGULIÈRES

Méta caractères

- ▶ `.` : Correspond à n'importe quel caractère sauf un retour à la ligne.
- ▶ `^` : Indique le début d'une ligne.
- ▶ `$` : Indique la fin d'une ligne.
- ▶ `*` : Correspond à zéro ou plusieurs occurrences du caractère précédent.
- ▶ `+` : Correspond à une ou plusieurs occurrences du caractère précédent
- ▶ `?` : Correspond à zéro ou une occurrence du caractère précédent.
- ▶ `\` : Sert à échapper un métacaractère pour qu'il soit interprété comme un littéral.

EXPRESSIONS RÉGULIÈRES

Classes de caractères

- ▶ **[abc]** : Correspond à "a", "b", ou "c";
- ▶ **[a-z]** : Correspond à n'importe quelle lettre minuscule de "a" à "z" ;
- ▶ **[A-Z]** : Correspond à n'importe quelle lettre majuscule de "A" à "Z " ;
- ▶ **[0-9]** : Correspond à n'importe quel chiffre de 0 à 9
- ▶ **[^abc]** : Correspond à n'importe quel caractère sauf "a", "b", ou "c"

EXPRESSIONS RÉGULIÈRES

Quantificateurs

- ▶ *: Correspond à zéro ou plusieurs occurrences
- ▶ +: Correspond à un ou plusieurs occurrences
- ▶ ?: Correspond à zéro ou une occurrence.
- ▶ {n}: Correspond exactement à n occurrences
- ▶ {n, }: Correspond à au moins n occurrences
- ▶ {n,m}: Correspond à entre n et m occurrences

EXPRESSIONS RÉGULIÈRES

Classes de caractères prédéfinis

- ▶ **\d**: Correspond à n'importe quel chiffre, équivalent à `[0-9]`;
- ▶ **\D**: Correspond à un ou plusieurs occurrences
- ▶ **\w**: Correspond à n'importe quel caractère alphanumérique ou underscore, équivalent à `[a-zA-Z0-9_]`.
- ▶ **\W**: Correspond à n'importe quel caractère qui n'est pas alphanumérique ou underscore, équivalent à `[^a-zA-Z0-9_]`
- ▶ **\s**: Correspond à n'importe quel espace blanc (espace, tabulation, retour à la ligne)
- ▶ **\S**: Correspond à n'importe quel caractère qui n'est pas un espace blanc

ANALYSE DES DONNÉES

Données

- Options ;
- Motif(s) recherché(s) ;
- Nom(s) du (des) fichier(s)

ANALYSE DES DONNÉES

Sorties

Généralement ce sont les lignes qui contiennent le motif recherché qui sont affichées, mais dépendamment de l'option utilisée, la sortie varie :

- Le nombre de lignes qui contiennent le fichier (-c)
- Les lignes numérotées (-n);
- Les lignes qui ne contiennent pas le motif recherché (-v)

SCÉNARIOS DES CAS D'UTILISATION

Premier cas : Rechercher un motif dans un fichier (pas d'option)

Description: permet à l'utilisateur de rechercher un motif dans un fichier qui devront être fournis par l'utilisateur

Acteurs impliqués: utilisateur

Résultat: les lignes du fichier qui contiennent le motif sont affichées OU une erreur est signalée si la syntaxe de la ligne de commande n'est pas correcte (nombre d'arguments insuffisant, mauvais orthographe de la commande, fichier introuvable)

Syntaxe : mygrep « motif » file.txt

SCÉNARIOS DES CAS D'UTILISATION

Deuxième cas : Compter les occurrences d'un motif dans un fichier (option -c)

Description: permet à l'utilisateur de déterminer le nombre de lignes dans lesquelles apparaît un motif dans un fichier **Acteurs impliqués:** utilisateur

Résultat: JUSTE le nombre de lignes affiché OU une erreur est signalée si la syntaxe de la ligne de commande n'est pas correcte (nombre d'arguments insuffisant, mauvais orthographe de la commande, fichier introuvable)

Syntaxe : mygrep -c « motif » file.txt

SCÉNARIOS DES CAS D'UTILISATION

Troisième cas : Recherche insensible à la casse (option -i)

Description: permet à l'utilisateur de rechercher un motif dans un fichier sans prendre en compte la casse (majuscule ou minuscule ou mixte).

Acteurs impliqués: utilisateur

Résultat: les lignes du fichier qui contiennent le motif (quelque soit sa casse) sont affichées OU une erreur est signalée si la syntaxe de la ligne de commande n'est pas correcte (nombre d'arguments insuffisant, mauvais orthographe de la commande, fichier introuvable)

Syntaxe : `mygrep -i « motif » file.txt`

SCÉNARIOS DES CAS D'UTILISATION

Quatrième cas : Rechercher récursive (option -r)

Description: permet à l'utilisateur de rechercher un motif dans un fichier (pas d'option dans ce cas) qui devront être fournis pas l'utilisateur

Acteurs impliqués: utilisateur

Résultat: les lignes du fichier qui contiennent le motif affichées OU une erreur est signalée si la syntaxe de la ligne de commande n'est pas correcte (nombre d'arguments insuffisant, mauvais orthographe de la commande, fichier introuvable)

Syntaxe : mygrep « motif » file.txt

SCÉNARIOS DES CAS D'UTILISATION

Cinquième cas : Rechercher un motif en expression régulière dans un fichier (option -G)

Description: permet à l'utilisateur de rechercher un motif complexe dans un fichier qui devront être fournis par l'utilisateur

Acteurs impliqués: utilisateur

Résultat: les lignes du fichier qui contiennent le motif recherché sont affichées
OU une erreur est signalée si la syntaxe de la ligne de commande n'est pas correcte (nombre d'arguments insuffisant, mauvais orthographe de la commande, fichier introuvable, expression régulière complexe)

Syntaxe : mygrep -G « motif » file.txt

SCÉNARIOS DES CAS D'UTILISATION

Sixième cas : Rechercher un motif (chaîne fixe) dans un fichier (option -F)

Description: permet à l'utilisateur de rechercher un motif dans un fichier qui devront être fournis par l'utilisateur

Acteurs impliqués: utilisateur

Résultat: les lignes du fichier qui contiennent le motif sont affichées OU une erreur est signalée si la syntaxe de la ligne de commande n'est pas correcte (nombre d'arguments insuffisant, mauvais orthographe de la commande, fichier introuvable)

Syntaxe : mygrep -F « motif » file.txt

SCÉNARIOS DES CAS D'UTILISATION

Septième cas : Affichage du contexte (-A, -B, -C)

Description: Permet d'afficher les lignes autour d'une correspondance

Acteurs impliqués: utilisateur

Déclencheur: mygrep -A 2 « hello » file.txt

SCÉNARIOS DES CAS D'UTILISATION

Septième cas : Affichage du contexte (-A, -B, -C)

Scénario nominal

1. Une correspondance est trouvée. 2. La ligne contenant le motif est affichée. 3. Les lignes de contexte sont affichées.

Variantes

Afficher les lignes après

`mygrep -A 2 hello file.txt` Affiche : ligne trouvée ligne suivante 1 ligne suivante

SCÉNARIOS DES CAS D'UTILISATION

Septième cas : Affichage du contexte (-A, -B, -C)

Scénario nominal

1. Une correspondance est trouvée. 2. La ligne contenant le motif est affichée. 3. Les lignes de contexte sont affichées.

Variantes

Afficher les lignes avant

`mygrep -B 2 hello file.txt` Affiche : ligne précédente 1 ligne précédente
2 ligne trouvée

SCÉNARIOS DES CAS D'UTILISATION

Septième cas : Affichage du contexte (-A, -B, -C)

Scénarios alternatifs

A1 : Correspondance en début de fichier Impossible d'afficher toutes les lignes précédentes. Le programme affiche uniquement les lignes disponibles.

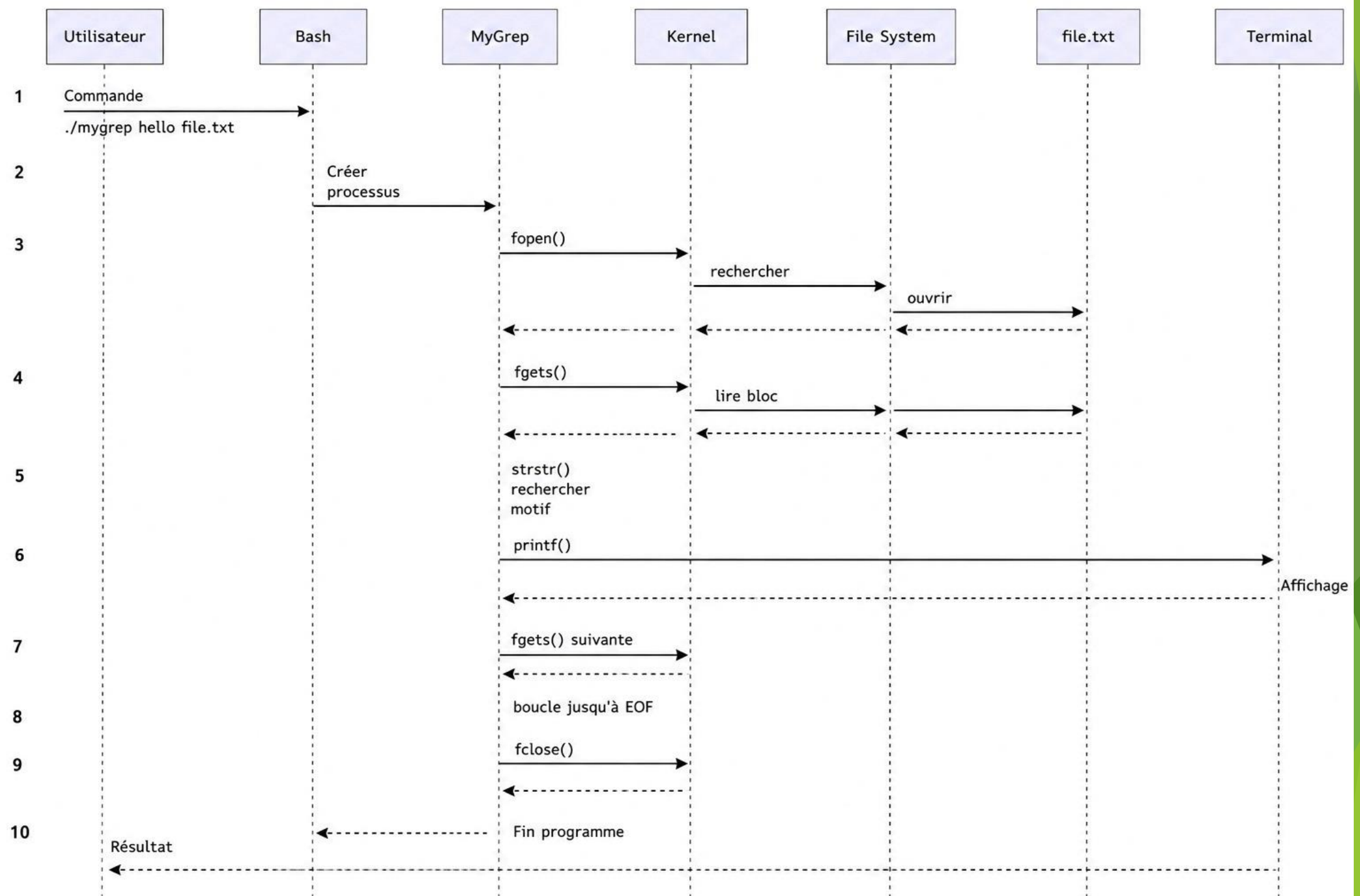
A2 : Correspondance en fin de fichier Impossible d'afficher toutes les lignes suivantes. Le programme affiche uniquement les lignes disponibles

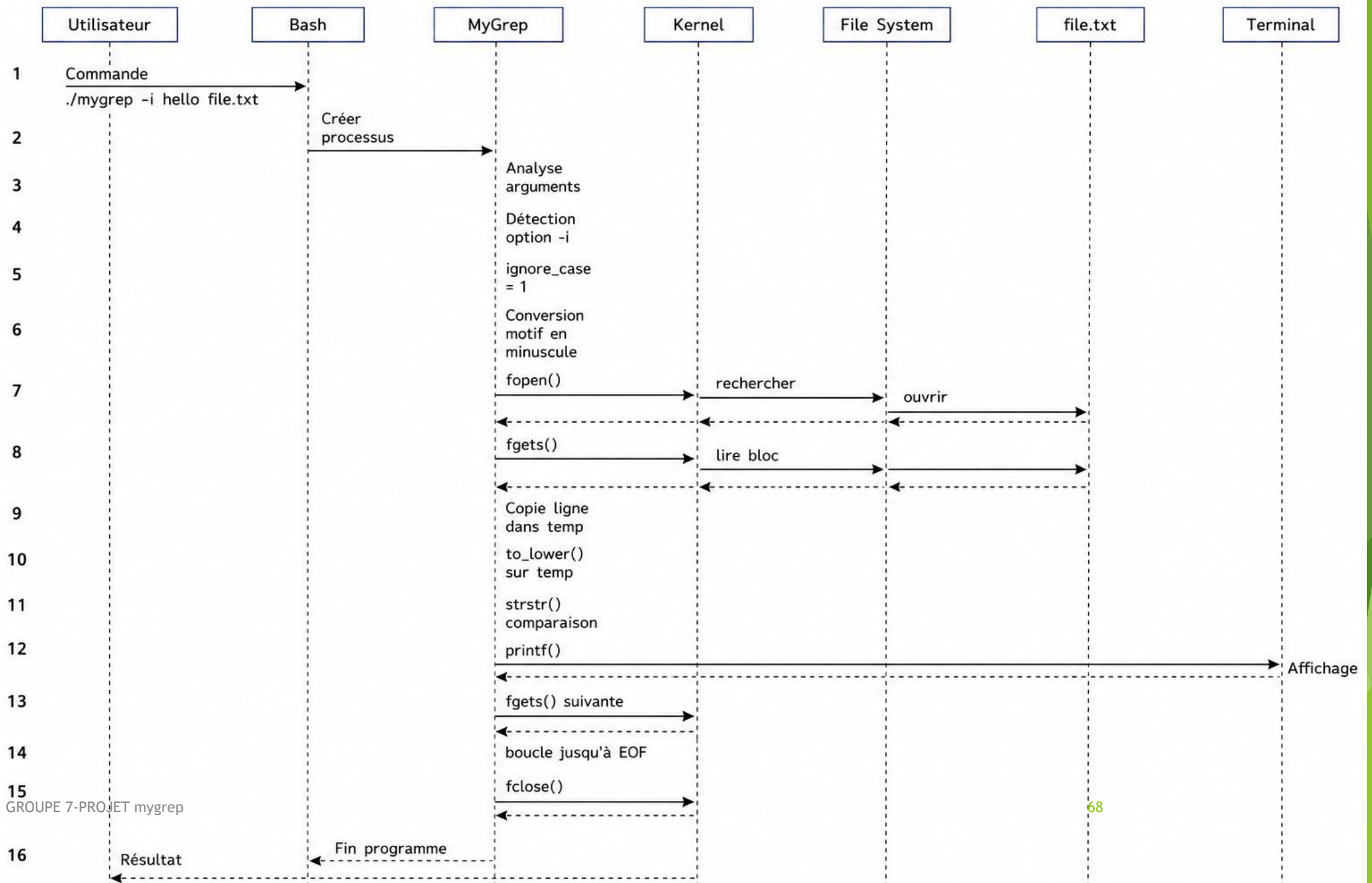
DOSSIER D'ANALYSE ET DE CONCEPTION

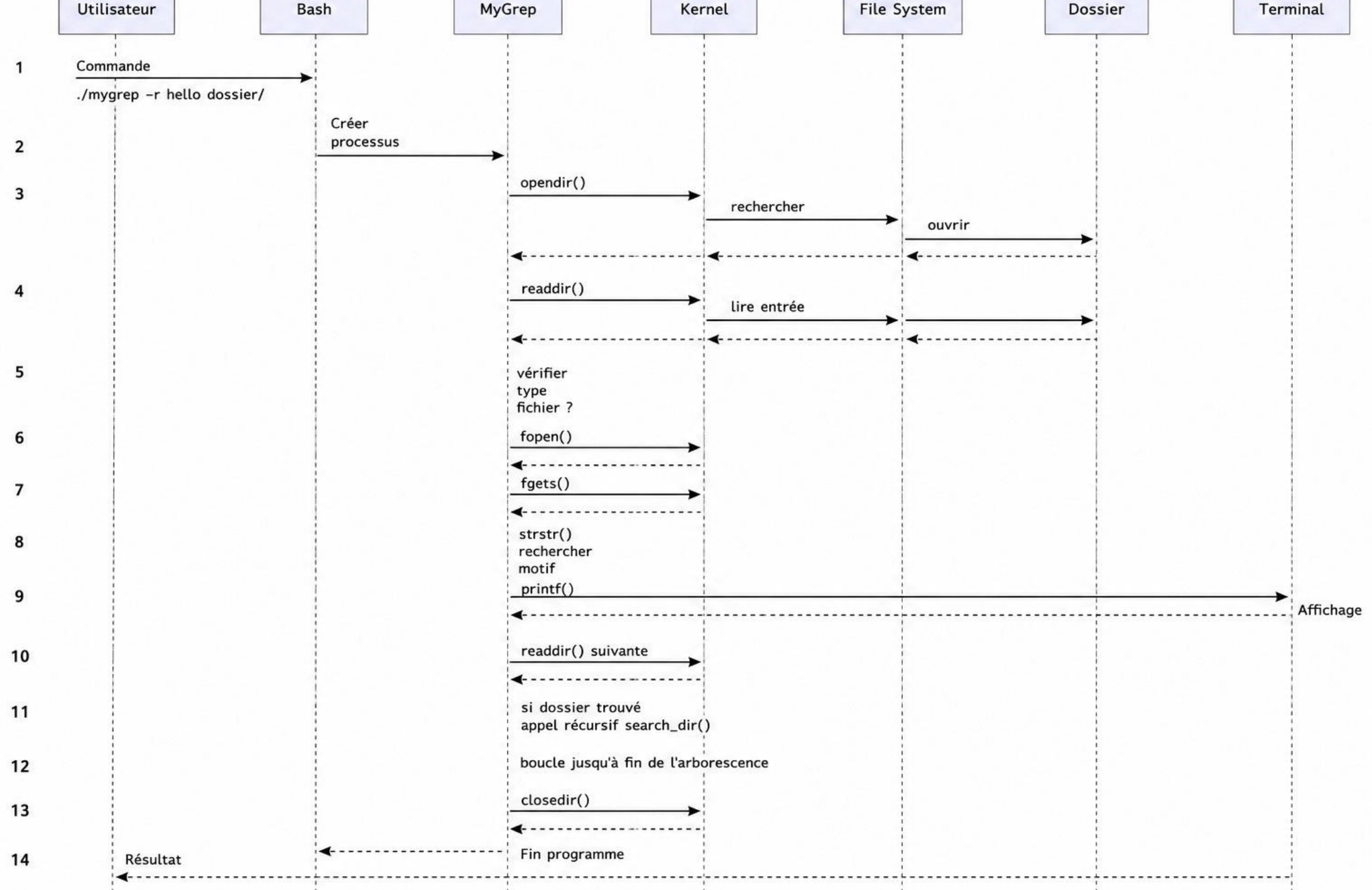
Conception

- ▶ Diagramme de séquences
- ▶ Contraintes techniques

DIAGRAMME DE SÉQUENCES







CONTRAINTES TECHNIQUES

- Langage de programmation
- Fonctionnement sur le système Linux

CONCLUSION

En définitive, effectuer ce projet nous a permis de comprendre le fonctionnement du système Linux à partir des commandes : les appels système, l'allocation de la mémoire. Bien que nous n'ayons pas pu implémenter toutes les fonctions, nous avons appris à travailler en équipe et apprendre ensemble.



INSTITUT UNIVERSITAIRE SAINT JEAN
SAINT JEAN INGÉNIEUR

RAPPORT TECHNIQUE

IMPLÉMENTATION DE LA COMMANDE GREP EN LANGAGE C

RAPPORT TECHNIQUE

- ▶ Introduction
- ▶ Déroulement du projet
- ▶ Justification des choix technologiques
- ▶ Résultats obtenus
- ▶ Difficultés rencontrées
- ▶ Perspectives et améliorations futures
- ▶ Lien du dépôt (repository) GitHub

INTRODUCTION

Ce rapport présente le déroulement du projet, les choix technologiques effectués, les résultats obtenus, les difficultés rencontrées ainsi que les perspectives d'amélioration.

DÉROULEMENT DU PROJET

Phase 1: Étude de la commande existante grep (étude du manuel, options), rédaction du cahier de charges et du dossier d'analyse;

Phase 2: apprentissage de git et GitHub, départage des tâches, rédaction du dossier de conception ;

Phase 3 : rédaction des fonctions associées aux options et tests unitaires ;

Phase 4: Combinaison des options et tests, rédaction du rapport technique ;

JUSTIFICATION DE L'ENVIRONNEMENT DE DÉVELOPPEMENT

Choix du langage de programmation : C

Justification : imposé par l'enseignant

JUSTIFICATION DE L'ENVIRONNEMENT DE DÉVELOPPEMENT

Choix de l'IDE : Visual Studio Code

Justification : cet IDE nous permettait d'éditer le code et à la fois de gérer les modifications sur GitHub avec les terminaux

RÉSULTATS OBTENUS

- Nous avons pu implémenter 36 options sur les 49 de la commande originale grep

DIFFICULTÉS RENCONTRÉES

- Apprivoisement de git et GitHub ;
- Gestion de l'allocation de la mémoire ;
- Manipulation des fichiers en langage C ;
- Initiation à la programmation système ;
- Implémentation des options gérant les expressions régulières ;
- Le crash des machines virtuelles ;
- La baisse des performances des machines hôtes ;

PERSPECTIVES ET AMÉLIORATIONS FUTURES

- Utilisation des pointeurs à la place des tableaux pour une meilleure gestion de la mémoire ;
- Implémentation des autres options ;

LIEN DU DÉPOT GITHUB

<https://github.com/OpenSecureFoundation/mygrep>